🏦 Apex Bank — Complete Project Guide & Progress Tracker

> **Purpose:** Yeh file is project ki poori learning, decisions, aur progress track karti hai. Naye Claude session mein is file ko paste karo — context instantly restore ho jayega.

📌 Project Overview

Project Name: Apex Bank
Goal: Production-grade digital banking backend — recruiters ko directly offer letter mile itna impressive
Approach: Learning while building (design → build → scale)
Timeline: 6 months
GitHub: https://github.com/afsar-hussai/Apex-Bank

🛠 Final Tech Stack (Decided)

Layer	Technology	Reason
Backend	**Go (Golang)**	150k+ req/s, goroutines, compiled binary, type safety
HTTP Framework	**Gin / Echo**	Fast, minimal, production-ready
Primary DB	**PostgreSQL**	ACID compliance — non-negotiable for banking
Cache	**Redis**	Sessions, balance cache, rate limiting
Message Queue	**Kafka**	Async events — notifications, fraud detection
Inter-service	**gRPC + protobuf**	Faster than REST for internal calls
Migrations	**golang-migrate**	SQL migration management
DB Queries	**sqlc + pgx**	Type-safe SQL in Go
Auth	**JWT (golang-jwt) + bcrypt**	Stateless auth, secure passwords
Containers	**Docker**	Multi-stage builds — 10MB Go image
Orchestration	**Kubernetes**	HPA, deployments, health probes
CI/CD	**GitHub Actions**	go test → go vet → golangci-lint → deploy
Monitoring	**Prometheus + Grafana**	Metrics, dashboards, alerts
Tracing	**Jaeger / OpenTelemetry**	Distributed request tracing

Layer	Technology	Reason
Logging	**Uber Zap**	Structured logging
Frontend	**Next.js + Tailwind CSS**	Customer portal + Admin dashboard

---

🏗️ Architecture Decisions (With Reasons)

Monolith First, Microservices Later

- **Phase 1–3:** Ek monolith — sab ek codebase mein
- **Phase 4+:** Services alag karo — Notification pehle, phir baaki
- **Kyun:** Code delete nahi hota — sirf move hota hai. Service boundaries khud samajh aati hain banate waqt. Twitter, Uber, Netflix — sab monolith se shuru kiye.

Architecture Layers (5-Tier)

```
1. Presentation Layer    — Next.js (Customer App + Admin Dashboard)
2. API Gateway           — Single entry point, rate limiting, auth routing
3. Business Logic Layer  — Auth, Account, Payment, Notification Services
4. Cache Layer           — Redis (sessions, balance cache)
5. Data Layer            — PostgreSQL (permanent storage)
```

Database Strategy

- **Phase 1–2:** Ek shared PostgreSQL — sab services use karein
- **Phase 3+:** Har microservice ka apna PostgreSQL
- **Notification Service:** MongoDB (flexible schema, no ACID needed)
- **Sessions/Cache:** Redis
- **Fraud Detection:** Redis (real-time, microseconds)
- **Kyun PostgreSQL for financial data:** ACID guarantee — partial transaction impossible

---

📋 Requirements (Finalized)

Functional Requirements

- User Registration with KYC
- Secure Login — Email/Password + 2FA
- Role-Based Access — Customer, Teller, Admin

- Multiple Account Types — Savings, Current, Fixed Deposit
- Real-Time Balance Updates
- Account Statement — PDF Download
- Fund Transfer — NEFT/RTGS/IMPS Simulation
- Idempotent Transactions — Double Payment Prevention
- Transaction History — Searchable, Paginated
- Real-Time Notifications — Email + In-App
- Basic Fraud Detection
- Account Freeze/Unfreeze
- Admin Dashboard
- Audit Logs

Non-Functional Requirements

- Transaction Latency: < 500ms P99
- Scale: 10M users, 100k concurrent requests
- Throughput: 10,000 TPS
- Uptime: 99.99%
- ACID Compliant Transactions
- Disaster Recovery: RTO 15min, RPO 1min
- Encryption: AES-256 at rest, TLS 1.3 in transit
- Rate Limiting + DDoS Protection
- Monitoring: Prometheus + Grafana
- Distributed Tracing: Jaeger

🗺 6-Month Roadmap

Phase 1 — Go Fundamentals + Project Setup (Month 1–2)

- [ ] Go language: goroutines, channels, interfaces, error handling, context
- [ ] Clean Architecture: handler → service → repository → DB
- [ ] Project folder structure setup
- [ ] PostgreSQL + Redis local setup (Docker)
- [ ] First API: User Registration + Login with JWT

Phase 2 — Core Banking Engine (Month 2–3)

- [ ] Double-entry ledger system
- [ ] Account types: Savings, Current, Fixed Deposit

- [ ] Fund transfer with ACID transactions
- [ ] Idempotency keys implementation
- [ ] NEFT/RTGS/IMPS simulation with worker pools

Phase 3 — Microservices + Event System (Month 3–4)

- [ ] Separate: Auth, Account, Payment, Notification, KYC services
- [ ] gRPC + protobuf inter-service communication
- [ ] Kafka integration — SAGA pattern
- [ ] API Gateway with middleware

Phase 4 — DevOps + Kubernetes (Month 4–5)

- [ ] Docker multi-stage builds
- [ ] Kubernetes: deployments, HPA, ConfigMaps
- [ ] GitHub Actions CI/CD pipeline
- [ ] Prometheus metrics + Grafana dashboards

Phase 5 — Scaling + Security (Month 5–6)

- [ ] PostgreSQL read replicas + connection pooling (pgxpool)
- [ ] Redis distributed locks + caching strategies
- [ ] AES-256-GCM encryption
- [ ] mTLS between services
- [ ] Fraud detection goroutines

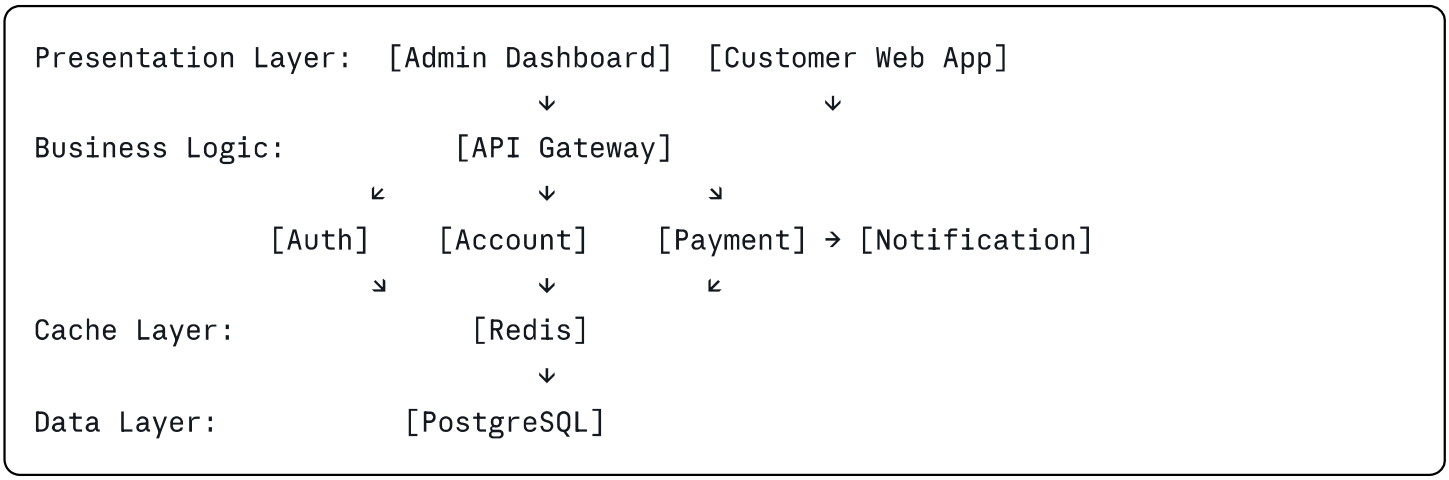
Phase 6 — Advanced Features (Month 6+)

- [ ] Loan/EMI engine
- [ ] Admin dashboard (Next.js)
- [ ] WebSocket real-time notifications
- [ ] Load testing with k6
- [ ] Database sharding (if needed at scale)

📐 Diagrams Completed

Diagram	Status	File
HLD (High Level Design)	✅ Done	`/Images/Architecture.png`
Data Flow Diagram	✅ Done	`/Images/Data_Flow.png`
DB Schema	🔄 In Progress	dbdiagram.io
Go Folder Structure	⬜ Pending	—

HLD Summary



Data Flow — Fund Transfer

```
Customer App → API Gateway → Auth Service (token verify)
→ Account Service (balance check → Redis cache → PostgreSQL fallback)
→ PostgreSQL (debit sender) → PostgreSQL (credit receiver) [SAME TRANSACTION]
→ Payment Service (record + idempotency key)
→ Notification Service (async alert)
→ Customer App (response)
```

📁 Go Project Structure (Planned)

```
apex-bank/
├── cmd/
│   └── server/
│       └── main.go              # Entry point
├── internal/
│   ├── auth/
│   │   ├── handler.go           # HTTP handlers
│   │   ├── service.go           # Business logic
│   │   └── repository.go        # DB queries
│   ├── account/
│   │   ├── handler.go
│   │   ├── service.go
│   │   └── repository.go
│   ├── payment/
│   │   ├── handler.go
│   │   ├── service.go
│   │   └── repository.go
│   └── notification/
│       ├── handler.go
```

```
│       ├── service.go
│       └── repository.go
├── pkg/
│   ├── errors/                 # Custom error types
│   ├── logger/                 # Zap structured logging
│   └── validator/              # Request validation
├── migrations/                 # SQL migration files
├── docker-compose.yml
├── Makefile
├── go.mod
└── README.md
```

🗄️ DB Schema (Next Step — dbdiagram.io)

Users Table (Start Here)

sql

```
Table users {
  id uuid [pk, default: `gen_random_uuid()`]
  full_name varchar(100) [not null]
  email varchar(255) [unique, not null]
  phone varchar(20) [unique, not null]
  password_hash varchar(255) [not null]
  role varchar(20) [not null, default: 'customer']
  kyc_status varchar(20) [not null, default: 'pending']
  is_active boolean [not null, default: true]
  created_at timestamptz [not null, default: `now()`]
  updated_at timestamptz [not null, default: `now()`]
}
```

> Tables remaining: accounts, transactions, ledger_entries, notifications, audit_logs

---

🧠 Key Concepts Learned

Concept	Simple Explanation
Architecture	App andar se kaise bani hai — layers, organization
Monolith	Sab ek codebase mein — simple, fast to build
Microservices	Har feature alag service — independent deploy/scale
3-Tier	Presentation + Business Logic + Data — basic structure

Concept	Simple Explanation
N-Tier	Zaroorat ke hisaab se layers badhti hain (API GW, Cache)
HLD	High Level Design — boxes aur arrows ka overview diagram
ACID	Atomic, Consistent, Isolated, Durable — DB transaction guarantee
Idempotency	Same request baar baar bhejo — ek hi baar process ho
Goroutine	Go ka lightweight thread — 2KB, millions ek saath
Channel	Goroutines ke beech safe data passing
Interface	Go mein contract — testing ke liye mock banane deta hai
Context	Request ke saath timeout/cancellation travel karta hai
Sharding	Ek bada DB → kai chote DBs — load distribute karo
Redis Cache	Cache hit → fast return, Cache miss → DB se fetch
Event-Driven	Services queue se async communicate karti hain

💻 Two Laptop Git Workflow

```bash
# Har session SHURU mein — dono laptops pe
git pull origin main

# Kaam ke BAAD — dono laptops pe
git add .
git commit -m "descriptive message"
git push origin main

# Agar conflict aaye
git pull --rebase origin main
```

VS Code shortcut: Bottom-left sync button (circle icon) — pull + push dono ek click mein.

📚 Key Resources

Resource	Link	Purpose
Tech School (YouTube)	youtube.com/c/TECHSCHOOLGURU	Go + PostgreSQL banking backend
simplebank (GitHub)	github.com/techschool/simplebank	Architecture reference
System Design Primer	github.com/donnemartin/system-design-primer	Reference only — selectively padho
dbdiagram.io	dbdiagram.io	DB schema design
Excalidraw	excalidraw.com	Architecture diagrams
Go Tour	tour.golang.org	Go language basics

🤖 Claude Ko Best Use Karne Ki Tips

Naye Session Mein Context Restore Karna

Is file ka content copy karo aur Claude ko yeh likho:

```
"Yeh mera Apex Bank project guide hai — [paste file content]
Main [specific task] karna chahta hoon."
```

Specific Requests Jo Best Results Deti Hain

```
# Code ke liye
"Apex Bank ke liye Go mein [feature] banao —
Clean Architecture follow karo, error handling include karo"

# Concept ke liye
"[Concept] ko Roman Urdu mein simple analogy se explain karo,
banking context mein example do"

# Review ke liye
"Yeh mera [code/diagram] hai — [screenshot/code paste karo]
Kya improvement chahiye? Kya galat hai?"

# Agle step ke liye
```

```
"Main abhi [current position] pe hoon.
Agla step kya hai aur kaise karoon?"
```

Kya Mat Karo

- Ek saath bahut saare sawaal mat poocho — ek ek karo
- Vague mat poocho — "help karo" ki jagah "X feature mein Y problem hai"
- Poora concept ek baar mein seekhne ki koshish mat karo

✅ Current Status

```
✅ Requirements.md          — Complete
✅ README.md                — Complete
✅ HLD Diagram              — Complete
✅ Data Flow Diagram        — Complete
🔄 DB Schema               — Next (dbdiagram.io pe users table se shuru)
⬜ Go Project Setup         — Phase 1
⬜ Auth Service             — Phase 1
⬜ Account Service          — Phase 2
⬜ Payment Service          — Phase 2
⬜ Notification Service     — Phase 2
⬜ DevOps Setup             — Phase 4
⬜ Microservices Migration  — Phase 3
⬜ Frontend                 — Phase 6
```

Last Updated: Architecture + Diagrams phase complete. Next: DB Schema on dbdiagram.io